

Failure (force the parser to actually fail) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

```
# structured_output_practice.py – Failure section
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

class Rating(BaseModel):
    score: int

structured_model = ChatOpenAI().with_structured_output(Rating)

try:
    structured_model.invoke("Describe your favorite color, no numbers at all.")
except Exception as e:
    print("Error type:", type(e))
    print("Error message:", e)
```

This version works and shows you the failure. Catching the broad `Exception` finds it, but doesn't tell your future code exactly which class to catch on purpose — you still have to go read the printed type by hand.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
# structured_output_practice.py – Failure section
from pydantic import BaseModel, ValidationError
from langchain_openai import ChatOpenAI

class Rating(BaseModel):
    score: int

def get_rating_chain() -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output(

def main() -> None:
    structured_model = get_rating_chain()

    try:
        structured_model.invoke("Describe your favorite color, no numbers at all.
    except ValidationError as e:
        print(f"Caught a ValidationError (expected): {e}")
    except Exception as e:
        print(f"Caught a different error than expected: {type(e).__name__}: {e}")
        print(f"Full class chain: {type(e).__mro__}")

if __name__ == "__main__":
    main()
```

Difference from Basic: Basic catches everything with one broad `except Exception`. Intermediate catches the *specific* class (`ValidationError`) it expects, with a broader fallback that also prints the full class chain — this is exactly the "check the real type before assuming" habit Doc01's Core Concepts describes, applied here for real.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a caller-friendly function, expected failure returns `None`

```
# structured_output_practice.py – Failure section
import logging
from pydantic import BaseModel, ValidationError
from langchain_openai import ChatOpenAI

logger = logging.getLogger(__name__)

class Rating(BaseModel):
    score: int

structured_model = ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output(Rating)

def get_rating(question: str) -> Rating | None:
    try:
        return structured_model.invoke(question)
    except ValidationError as e:
        logger.warning(f"Model reply didn't match Rating shape: {e}")
        return None
    except Exception as e:
        logger.error(f"Unexpected error type from structured output: {type(e).__name__}")
        raise
```

The design choice here: an expected parsing failure (`ValidationError`) is a normal, recoverable event — log it and return `None` , and let the caller decide what to do (retry with a clarified prompt, show a fallback message). An *unexpected* error type is genuinely different — it might mean the library changed its wrapping behavior, or something else broke entirely — so it's logged at a higher severity and re-raised, not silently swallowed.

Approach 2 — retry once with a clarified prompt before giving up

```
# structured_output_practice.py – Failure section
def get_rating_with_retry(question: str) -> Rating | None:
```

```
result = get_rating(question)
if result is not None:
    return result

clarified = question + " Answer with a single number from 1 to 10."
return get_rating(clarified)
```

This is a genuinely different design decision, not just more code: instead of treating a parsing failure as final, it tries once more with a prompt that more strongly constrains the model toward a valid answer. Worth doing only when there's a real chance the retry fixes it (a vague prompt) — not when the field is fundamentally unanswerable from the question (retrying won't invent a number that was never there).

Difference from Intermediate: Intermediate catches the right class and logs useful detail — good for a script you're debugging by hand. Advanced turns it into a function other code actually calls: Approach 1 gives the caller a clean `Rating | None` contract instead of an exception to catch every time, and Approach 2 adds a real recovery strategy (retry with a clarified prompt) instead of just giving up on the first failure.

Which one should you actually write? For debugging or a one-off script, the Intermediate version's specific-catch-plus-fallback is enough. For a function other code will actually call — inside a real feature — write Approach 1: a clean `Rating | None` return, expected failures logged and handled gracefully, unexpected ones logged loudly and re-raised so they don't disappear silently. Add Approach 2's retry only where a clarified prompt genuinely has a shot at succeeding; retrying blindly just delays an error that was never going to resolve.